

3a. Write a program to check request header for cookies.

Program:

server.js

```
const http = require('http');

http.createServer((req, res) => {

  const cookies = req.headers.cookie;

  res.writeHead(200, {'Content-Type': 'text/plain'});

  res.end(`Cookies from request: ${cookies || 'No cookies found'}`);

}).listen(3000, () => console.log('Server running on port 3000'));
```

Execution Steps:

1. In Terminal Type, mkdir <dir_name>
2. cd <dir_name>
3. npm init -y
4. create **server.js** file in VSCode and type the above code.
5. In terminal type, node server.js->Enter
6. goto Google Chrome and type in the URL-> localhost:3000->Enter.
7. To get the cookie information, Goto chrome settings->more tools->developer tools
8. Then, console-> type, **document.cookie="pavan"**->enter.
9. Refresh the chrome and see the user cookie information.

b. Write node.js program to print the car object properties, delete the second property and get length of the object.

Program:

carObject.js

```
let car = { make: "Toyota", model: "Corolla", year: 2022 };

console.log("Original Properties:", Object.keys(car));

delete car.model;

console.log("After Deletion:", car);

console.log("Length of object:", Object.keys(car).length);
```

Execution Steps:

1. create **carObject.js** file in VSCode and type the above code.
2. In terminal type, `cd <dir_name>` #goto the current directory created earlier for 3a
3. Then type, `node carObject.js` -> Enter
4. Now we can see the output in the terminal.

4 a. Read the data of a student containing usn, name, sem, year_of_admission from node js and store it in the mongodb.

b. For a partial name given in node js, search all the names from mongodb student documents created in Question(a)

Program:

Student.js

```
const { MongoClient } = require('mongodb');
const url = "mongodb://localhost:27017";
const client = new MongoClient(url);

async function run() {
  try {
    await client.connect();
    const db = client.db('college');
    const students = db.collection('st');

    // Part A: Store Student
    const studentData = { usn: "1BM20CS001", name: "Rahul Kumar", sem: 4,
      year_of_admission: 2020 };
    const studentData1 = { usn: "1BM20CS002", name: "Rahim", sem: 4,
      year_of_admission: 2020 };
    const studentData2 = { usn: "1BM20CS003", name: "Rakesh", sem: 4,
      year_of_admission: 2020 };
```

```
await students.insertOne(studentData);
await students.insertOne(studentData1);
await students.insertOne(studentData2);
```

```
console.log("Student added.");
```

// Part B: Search Partial Name (e.g., "Rah")

```
const partialName = "Rah";
const results = await students.find({ name: { $regex: partialName, $options: "i"
} }).toArray();
console.log("Search Results:", results);
} finally {
await client.close();
}
}
run().catch(console.dir);
```

Execution Steps:

1. create **student.js** file in VSCode and type the above code.
2. In the terminal->E:\BMSIT DATA\Mern lab\Backend> node student.js
3. Now we can see the output in the terminal.

5. Implement all CRUD operations on a File System using Node JS.

Program:

Crud.js

```
const fs = require('fs');
```

```
const filename = 'data.txt';
```

// Create / Update

```
fs.writeFileSync(filename, 'Hello, this is the initial text.\n');
```

```
console.log("File Created.");

// Read

let content = fs.readFileSync(filename, 'utf8');

console.log("Read Data:", content);

// Update (Append)

fs.appendFileSync(filename, 'This is appended text.');
```

```
console.log("File Updated.");

// Read Updated data

let updata = fs.readFileSync(filename, 'utf8');

console.log("Updated Data:", updata);

// Delete

fs.unlinkSync(filename);

console.log("File Deleted.");
```

Execution Steps:

1. create **crud.js** file in VSCode and type the above code.
2. In the terminal->E:\BMSIT DATA\Mern lab\Backend> node crud.js
3. Now we can see the output in the terminal.